**Input**: arr[] = [900, 940, 950, 1100, 1500, 1800], dep[] = [910, 1200, 1120, 1130, 1900, 2000]

[ (900, a) → 1
(910, d) → 0
940, a → 1
950, a → 2
1100, a → 3
1120, d → 2
1130, d → 1
1200, d → 0
1500, a → 1
1800, a → 2
1900, d → 1
2000, d → 0 ]

(timestamp, a/d)

TC: $O(n + n \log n + n)$
$= O(n \log n)$

AS: $O(n)$

arr: 1000 1200
dep: 1200 1500

o/p: 2

1000 a
[ 1200 a
1200 d ]
1500 d

---

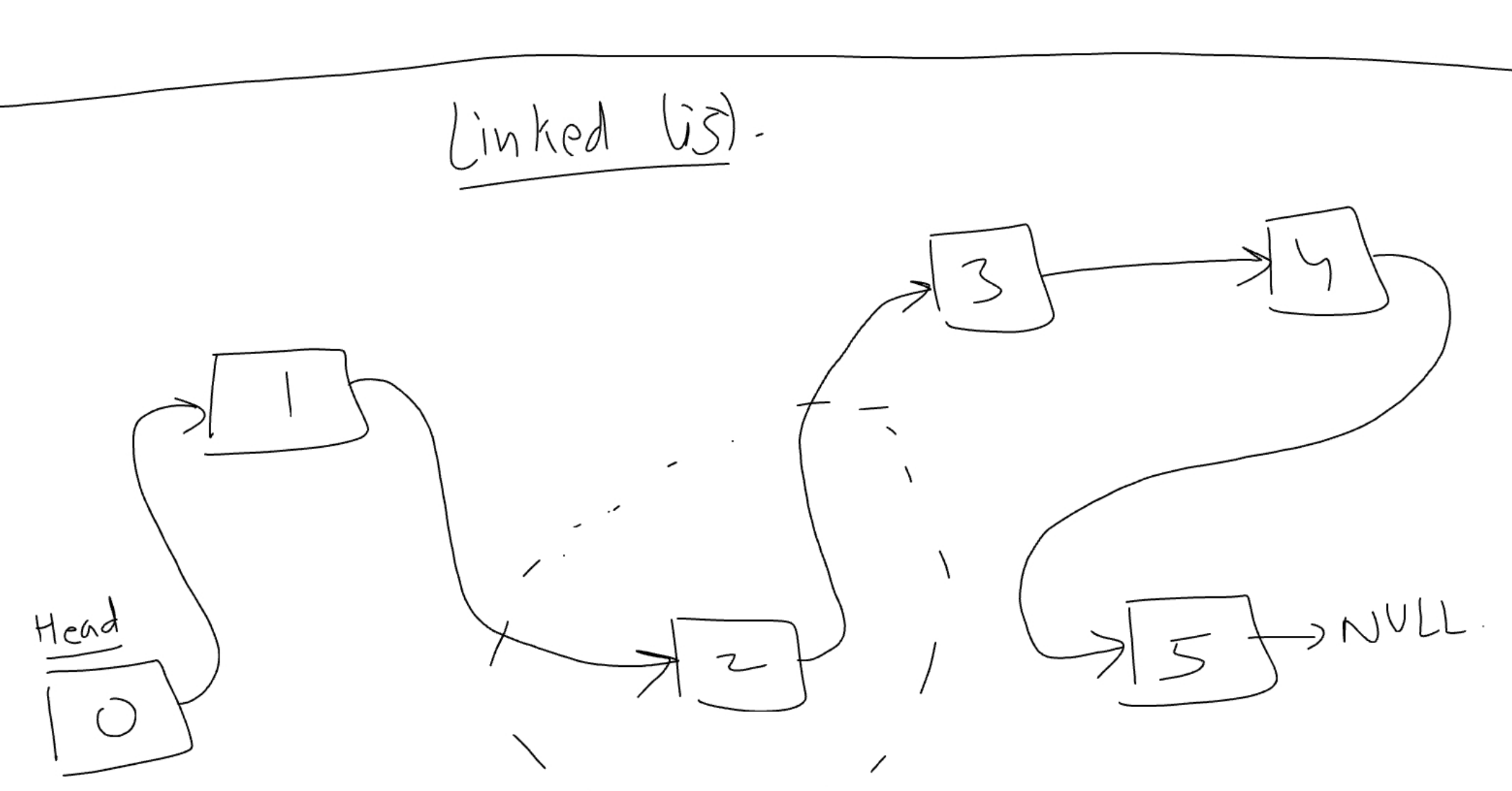
(1200, a) < (1200, d)

0000 2359

1 -1 1

---

Linked list.



Adv.

1) No requirement of large amt. of contiguous mem.
2) Insertion/deletion at beginning is easy.

Disadv.

1) No random access.
2) Not as cache friendly as arrays
3) storage overhead compared to arrays (for next)

Representation:

Node:

int data;
Node* next

---

i/p: Head 1 → 2 → 3 → N. , 0

o/p: Head 0 → 1 → 2 → 3 → N.

nn 0 → ~~head~~ 1 → 2 → 3 → N.
head

[ nn = new Node (0).
nn → next = head.
head = nn ]

head
0 → NULL

H
1 → 2 → 3 → N
t t t t

~~H 1 → N.~~
t != NULL

[1, 2, 3]

i/p: H 1 → 2 → 3 → N  t  Value = 4

o/p: H 1 → 2 → 3 → 4 → N.

H 1 → 3 → 3 → 4   3 → next = nn

→ [ nn = new Node(4).
t → next = nn ]

t t t
1 → 2 → 3 → N.

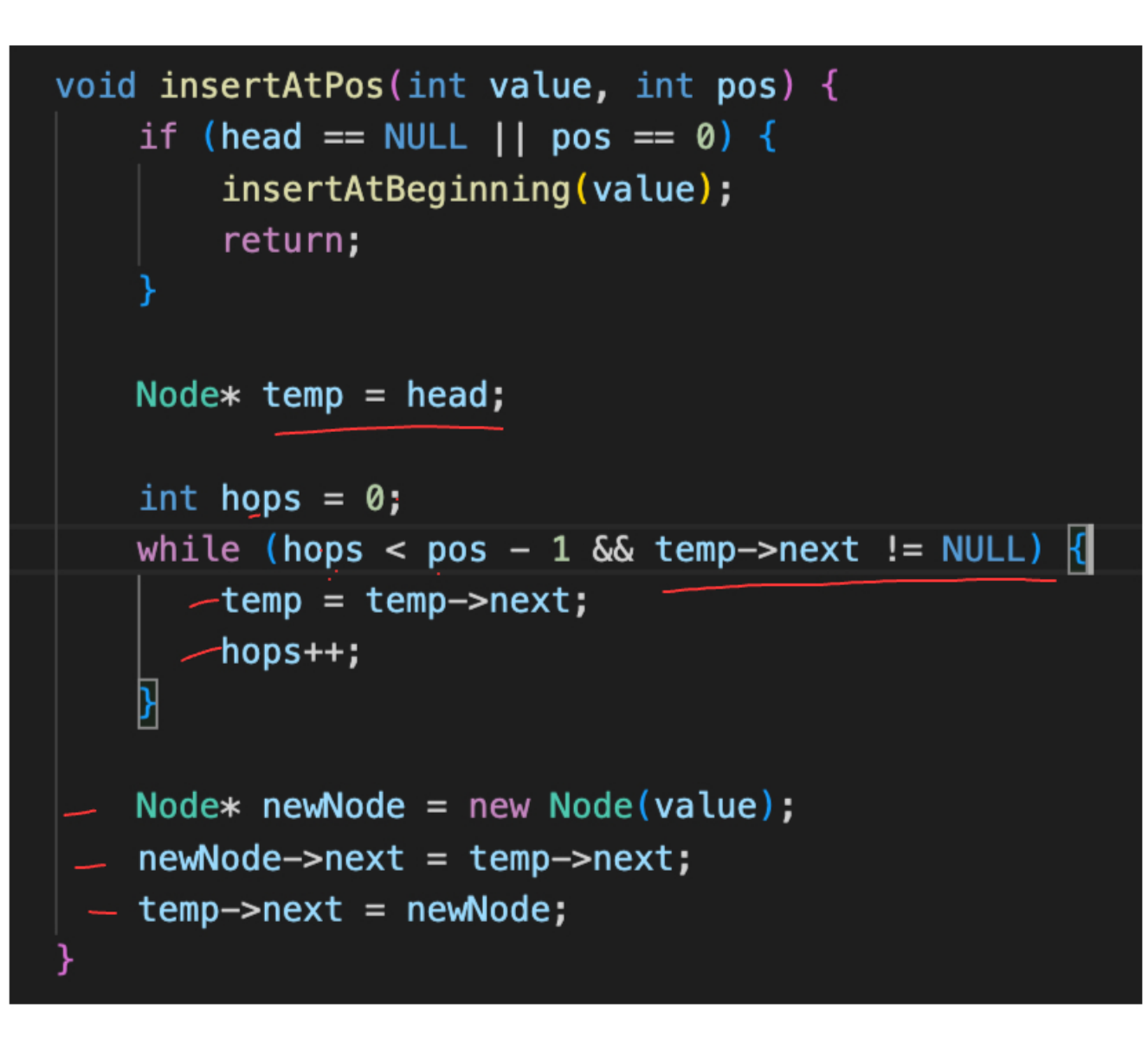
---

i/p: H 10 → 11 → 12 → 13 → 14 → N | value = 9 pos = 3

o/p: H 10 → 11 → 12 → 9 → 13 → 14 → N.

H 10 → 11 → 12 → 9 → 13 → 14 → N.
t t t

(pos-1) hops to reach 12.
[ nn → next = t → next
t → next = nn ].



```
void insertAtPos(int value, int pos) {
    if (head == NULL || pos == 0) {
        insertAtBeginning(value);
        return;
    }

    Node* temp = head;

    int hops = 0;
    while (hops < pos - 1 && temp->next != NULL) {
        temp = temp->next;
        hops++;
    }

    Node* newNode = new Node(value);
    newNode->next = temp->next;
    temp->next = newNode;
}
```

H
1 → 2 → 3 → ~~N~~ → 4 → N | Value = 4 pos = 100.
t t t

hops = ~~0~~ 1 2